Three-Tier Architecture Deployment

Three-Tier Architecture?

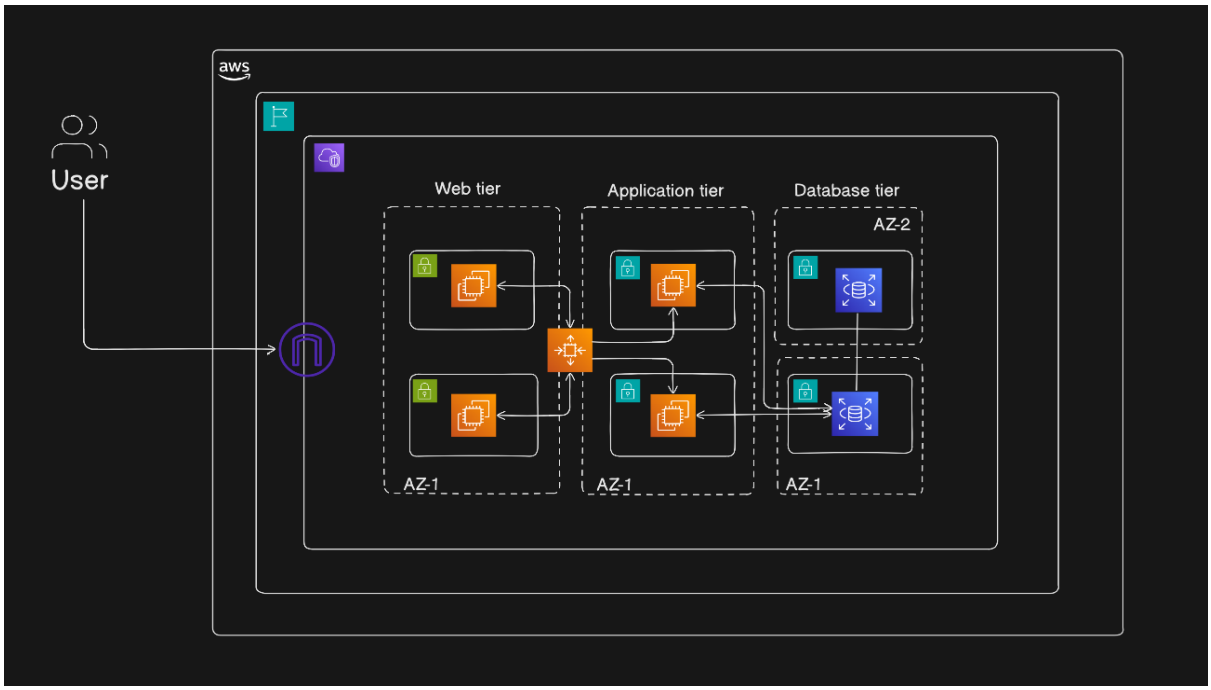
Three-Tier Architecture divides an application into **three logical layers**, each with a specific responsibility:

1. **Web Tier (Presentation Layer)**
   Handles incoming user requests and serves static content.
2. **Application Tier (Business Logic Layer)**
   Processes requests, applies business logic, and communicates with the database.
3. **Database Tier (Data Layer)**
   Stores and manages application data securely.

Objective:

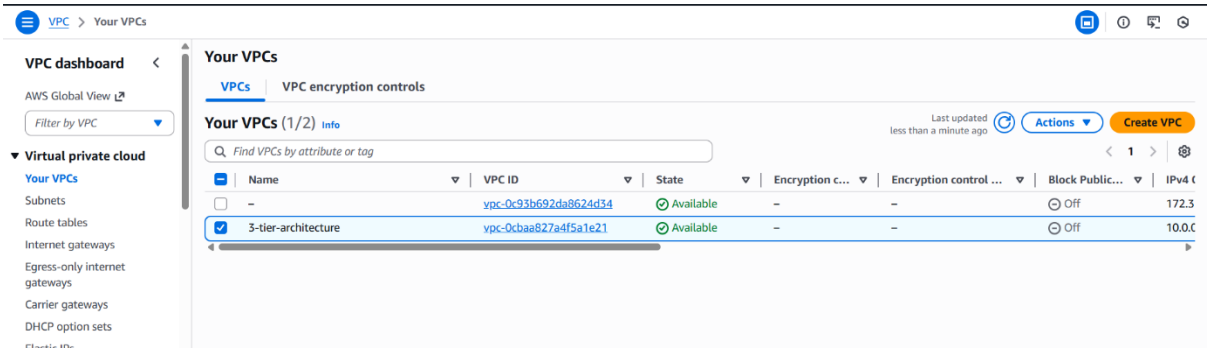
- Design and deploy a scalable Three-Tier Architecture on AWS.
- Implement a secure and highly available infrastructure using AWS services.
- Separate the application into **Web, Application, and Database tiers**.
- Deploy each tier in **isolated subnets** within a custom Amazon VPC.
- Use **public subnets** for the web tier and **private subnets** for application and database tiers.
- Apply AWS best practices for **networking, security, and fault tolerance**.
- Implement **Auto Scaling Groups** for automatic scaling and high availability.
- Use **Amazon RDS** for a managed and secure database layer.
- Configure a **NAT Gateway** to allow private instances outbound internet access.

Architecture



VPC

In this project, **Amazon VPC** is used to create a secure and isolated network environment for the three-tier architecture. The VPC provides a private IP address range (10.0.0.0/16) and enables the separation of web, application, and database layers into different subnets. This ensures controlled traffic flow, secure communication between tiers, and compliance with AWS best practices for scalability, security, and high availability.



Subnets

A **subnet** is a logical subdivision of an Amazon VPC that allows you to organize and isolate resources within a virtual network. Subnets help control network traffic by defining which resources can communicate with each other and with external networks. By placing resources in different subnets, organizations can improve security, manage routing efficiently, and design scalable and highly available architectures.

Subnets Created in This Project

Public Subnets (Web Tier)

- Two public subnets deployed across multiple Availability Zones.
- Used to host Web Tier EC2 instances.
- Associated with a route table that includes a route to the Internet Gateway.
- Enables inbound and outbound internet access.
- Supports high availability for user-facing components.

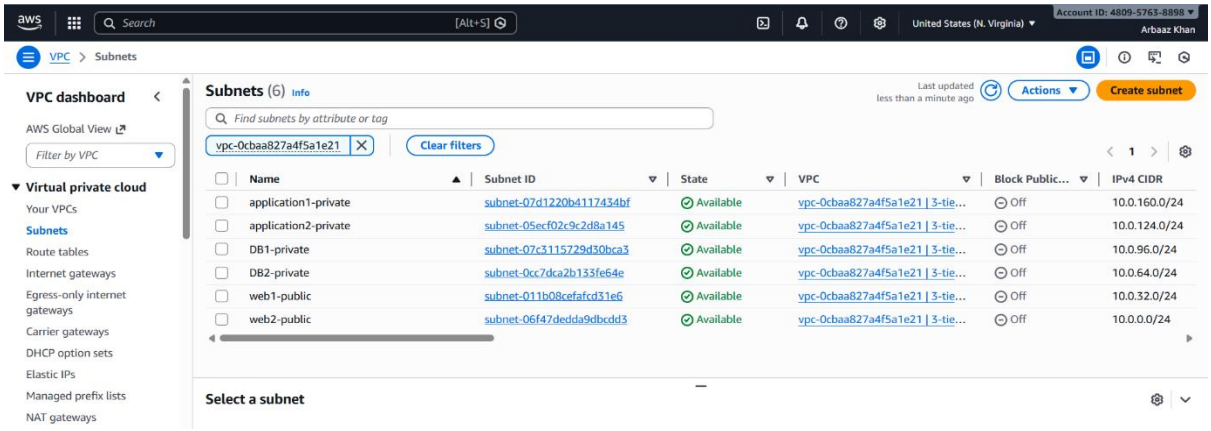
Private Subnets (Application Tier)

- Two private subnets created for the application layer.
- Used to host Application Tier EC2 instances.
- No direct internet access.
- Outbound access enabled using a NAT Gateway.
- Enhances security by isolating application logic.

Private Subnets (Database Tier)

- Two private subnets dedicated to the database layer.

- Used for Amazon RDS (MySQL).
- Fully isolated from the internet.
- Database access restricted to the application tier only.
- Ensures data security and compliance.



Route Tables

A **route table** is a set of rules, called routes, that determines how network traffic is directed within a VPC. Each subnet in a VPC must be associated with a route table, which controls where traffic is forwarded based on destination IP addresses. Route tables play a critical role in managing internet connectivity and controlling communication between different tiers of an application.

Route Tables Used in This Project

Public Route Table

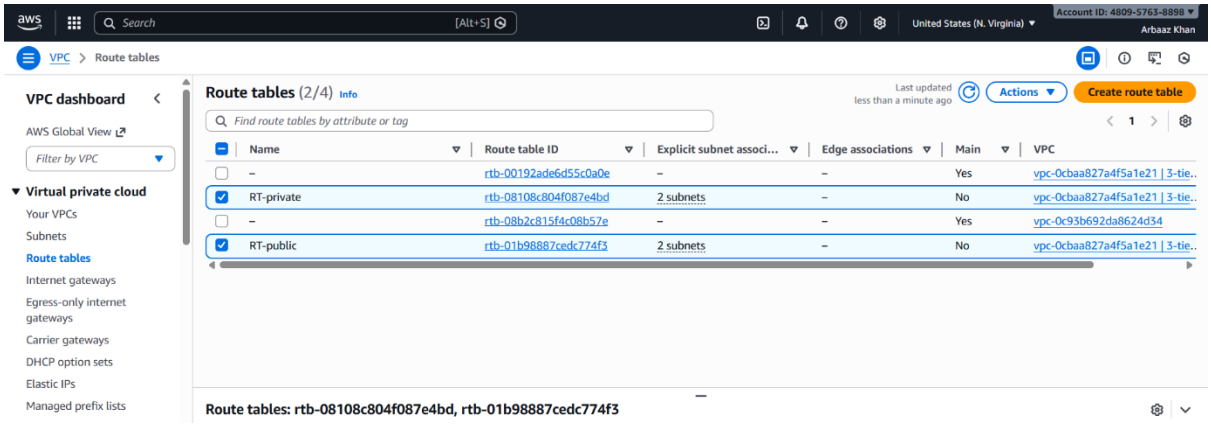
- Associated with public subnets hosting the web tier.
- Contains a route that directs all outbound internet traffic to the Internet Gateway.
- Route configuration:
  0.0.0.0/0 → Internet Gateway
- Allows web servers to receive and respond to internet requests.

Private Route Table

- Associated with private subnets used by the application tier.
- Contains a route that directs outbound internet traffic through a NAT Gateway.
- Route configuration:
  0.0.0.0/0 → NAT Gateway
- Ensures private instances can access the internet securely without being exposed.

Importance of Route Tables

- Control and restrict internet access based on subnet type.
- Prevent direct internet exposure of sensitive resources such as databases.
- Enable secure outbound connectivity for private resources.
- Enforce tier-based traffic separation following AWS best practices.

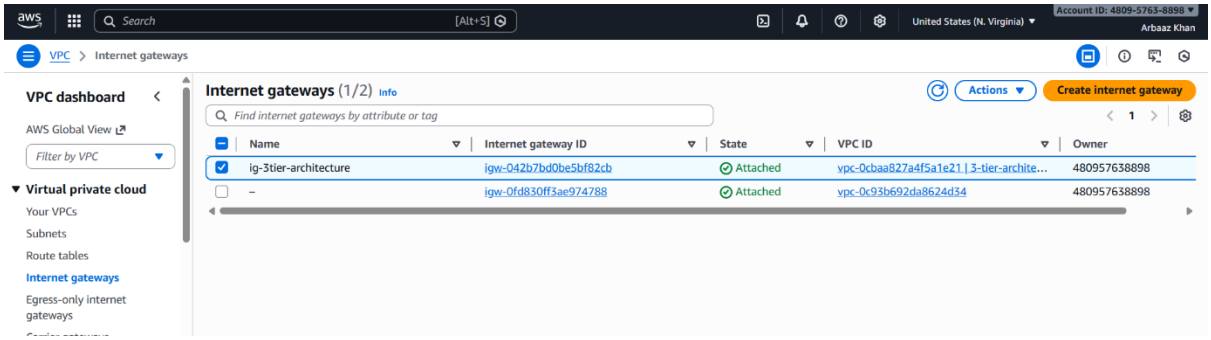


Internet Gateway (IGW)

An **Internet Gateway (IGW)** is a horizontally scaled, highly available AWS component that enables communication between resources in a VPC and the public internet. It allows instances in public subnets to send and receive traffic from the internet by providing a target for routes that direct outbound traffic.

Usage in This Project

- The Internet Gateway is **attached to the VPC**.
- It is referenced in the **public route table**.
- Enables **web-tier EC2 instances** in public subnets to:
    - Receive incoming user requests
    - Serve web content to the internet
- Ensures secure and controlled internet access for public-facing resources.

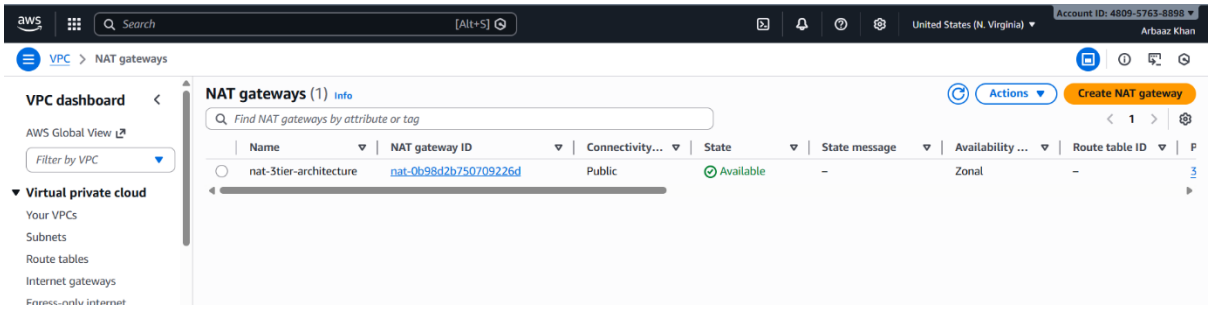


NAT Gateway

A **NAT Gateway (Network Address Translation Gateway)** is an AWS-managed service that enables instances in **private subnets** to initiate outbound connections to the internet while preventing inbound internet traffic from reaching them. This ensures that private resources can access external services securely without being publicly exposed.

Why NAT Gateway Is Needed in This Project

- **Application EC2 instances** are deployed in private subnets and do not have public IP addresses.
- These instances require outbound internet access for:
  - Installing operating system updates
  - Downloading application dependencies and packages
  - Accessing external APIs if required
- The NAT Gateway allows this outbound access while maintaining **network isolation and** security, ensuring the application tier remains inaccessible directly from the internet.



EC2 Instance Templates

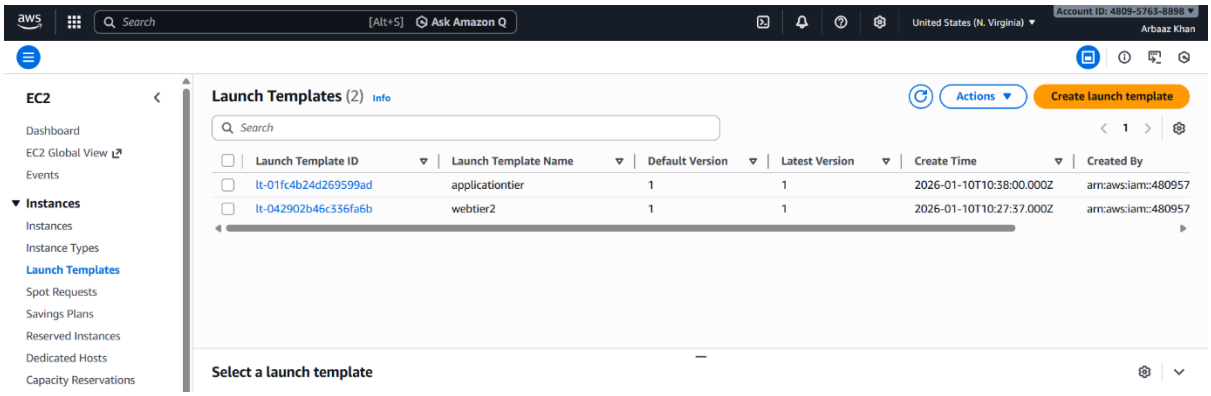
An **EC2 Instance Template (Launch Template)** is a reusable configuration blueprint that defines how EC2 instances should be launched. It standardizes instance creation by predefining settings such as the operating system, instance size, networking, and security. In this project, instance templates are used to ensure that all web and application servers are launched with identical and reliable configurations.

Included Configuration

- **Ubuntu AMI**
  Provides a stable and widely supported Linux operating system suitable for web and application workloads.
- **Instance Type (Free Tier)**
  Uses a cost-effective instance type (such as t2.micro or t3.micro) to stay within free-tier limits.
- **Security Group**
  Controls inbound and outbound traffic rules specific to each tier (web or application).
- **Key Pair**
  Enables secure SSH access for administration and troubleshooting.
- **User Data (Optional)**
  Automates initial setup tasks such as package installation or service startup during instance launch.

Why Instance Templates Are Used

- **Required for Auto Scaling Groups**
  Auto Scaling relies on launch templates to know how to create new instances automatically.
- **Ensures Uniform Configuration**
  Every instance launched follows the same configuration, reducing configuration drift.
- **Simplifies Scaling and Management**
  Updates can be managed by creating new template versions without manually reconfiguring instances.



Auto Scaling Groups (ASG)

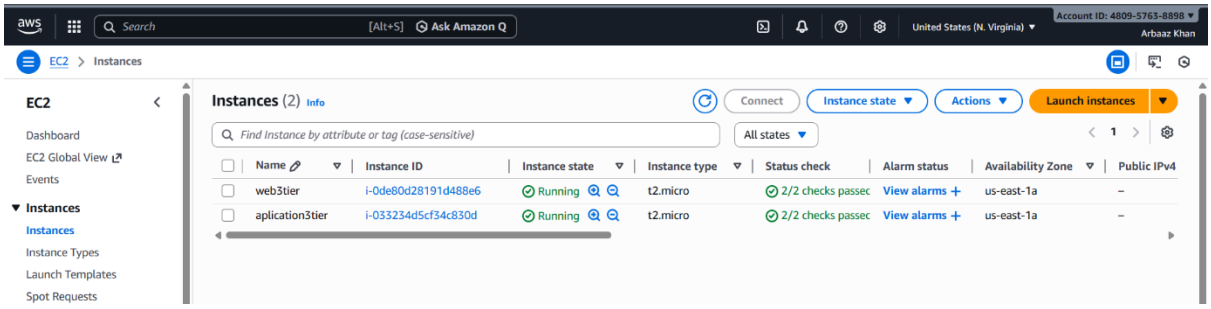
Auto Scaling is an AWS service that automatically adjusts the number of EC2 instances in response to traffic demand, system health, or predefined scaling policies. It ensures that the application always has the required capacity while minimizing manual intervention.

Usage in This Project

- **Web Tier Auto Scaling Group**
  Manages EC2 instances running the NGINX web server in public subnets. It automatically launches or terminates web servers based on load or health checks to ensure consistent availability to end users.
- **Application Tier Auto Scaling Group**
  Manages EC2 instances running the application logic (Python/Flask) in private subnets. It scales the application layer independently from the web layer, allowing better performance and isolation.

Benefits

- **High Availability**
  Instances are distributed across multiple Availability Zones, reducing downtime during failures.
- **Fault Tolerance**
  Automatically replaces unhealthy or terminated instances without manual action.
- **Automatic Recovery**
  Ensures the desired number of instances is always running, maintaining application stability.



Web Tier Deployment (Nginx)

The **Web Tier** is responsible for handling all incoming user requests and serving the frontend content of the application. In this project, **Nginx** is used as the web server because it is lightweight, high-performance, and well suited for serving static web content such as HTML and CSS files.

Role of Nginx in the Web Tier

- Acts as the **web server** for the application.
- Serves **static content** such as HTML and CSS files to users.
- Listens for and handles **incoming HTTP requests** on port 80.
- Provides fast and reliable content delivery for the presentation layer.

Commands Used for Nginx Installation and Management

sudo apt update

Updates the package repository to ensure the latest versions of software are available.

sudo apt install nginx -y

Installs the Nginx web server on the Ubuntu EC2 instance.

sudo systemctl start nginx

Starts the Nginx service so it can begin handling web requests.

sudo systemctl enable nginx

Configures Nginx to start automatically when the instance reboots.

Website Deployment Steps

cd /var/www/html

sudo rm index.debian_index.html

Navigates to the default web root directory used by Nginx.

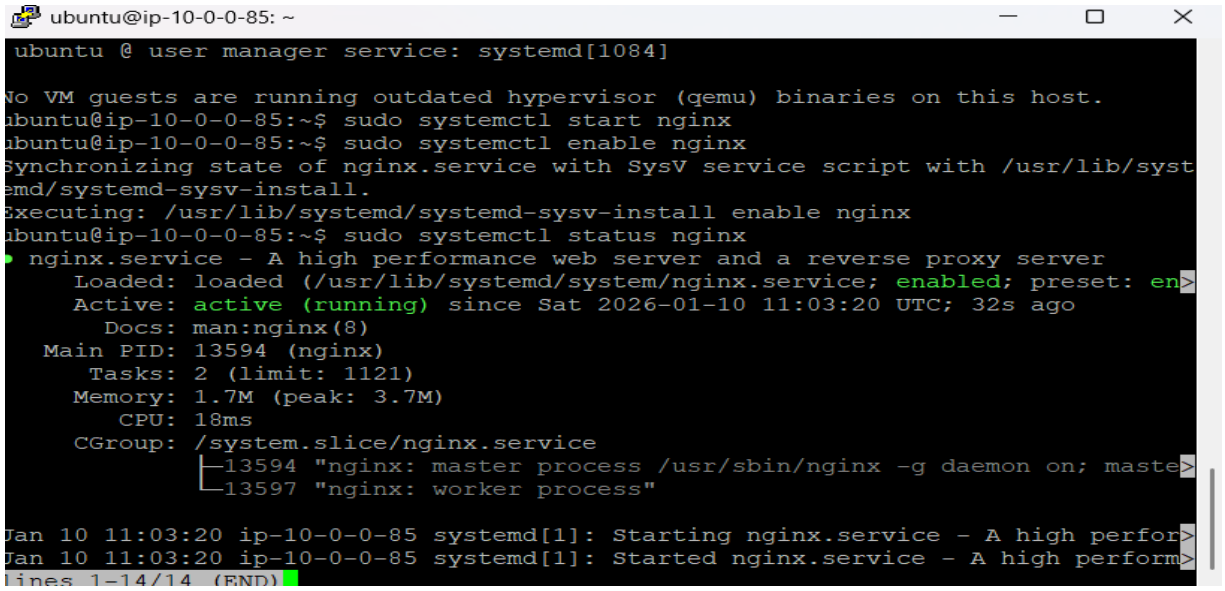
sudo nano index.html

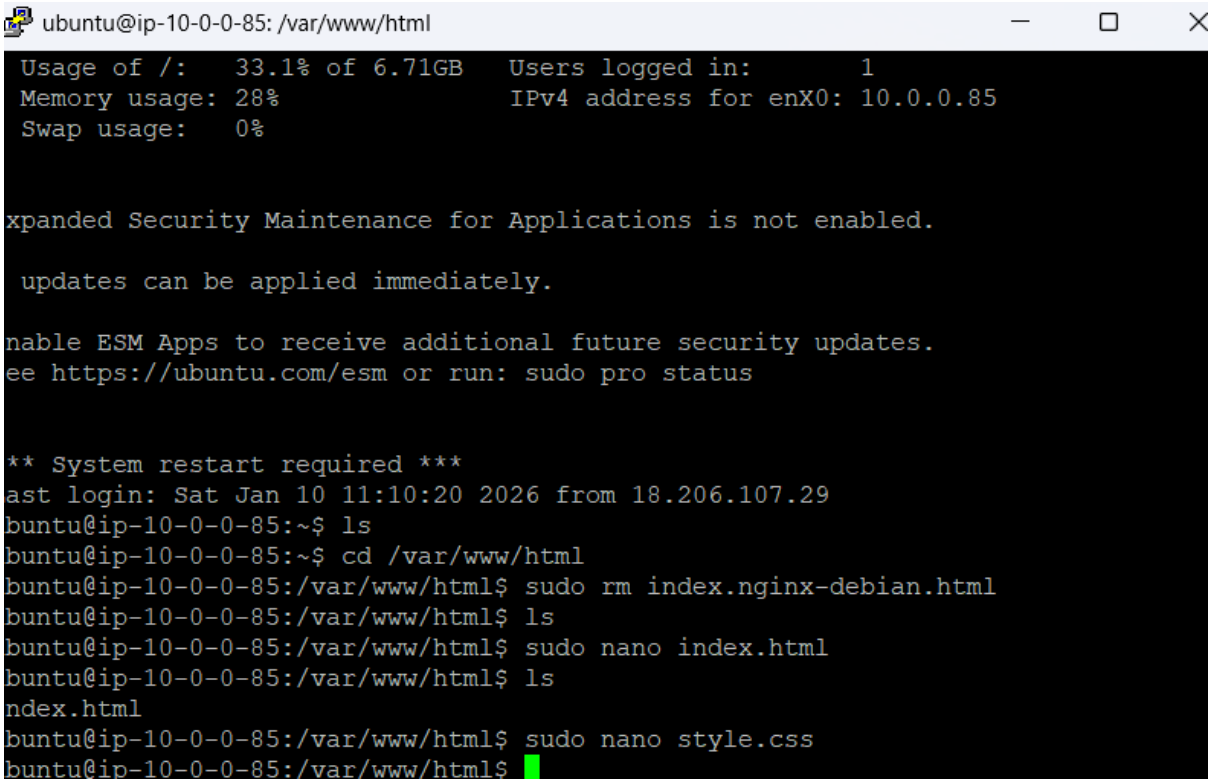
sudo nano style.css

Creates or edits the main website file containing HTML and CSS content.

sudo systemctl restart nginx

Restarts the Nginx service to apply the updated website content.





Application Tier Deployment (Python + Flask)

The **Application Tier** contains the core backend logic of the system. In this project, **Python with Flask** is used to build the application layer, which processes requests received from the web tier and communicates securely with the database tier.

Role of Flask in the Application Tier

- Acts as the **backend application framework**.
- Implements **business logic** and application rules.
- Handles requests forwarded from the web tier.
- Establishes communication with the **database tier (Amazon RDS – MySQL)**.
- Generates responses that are returned to the web tier.

Commands Used for Application Setup

sudo apt install python3 python3-pip -y

Installs Python 3 and the Python package manager (pip), which are required to run and manage the Flask application.

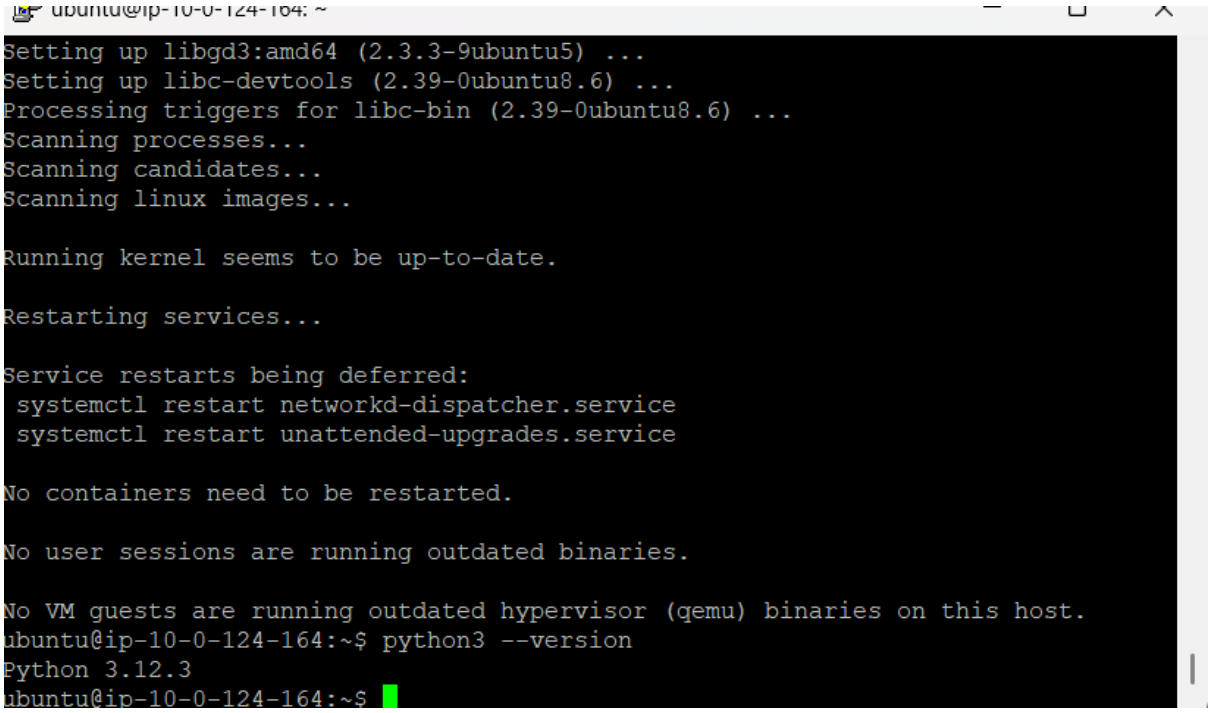
pip3 install flask

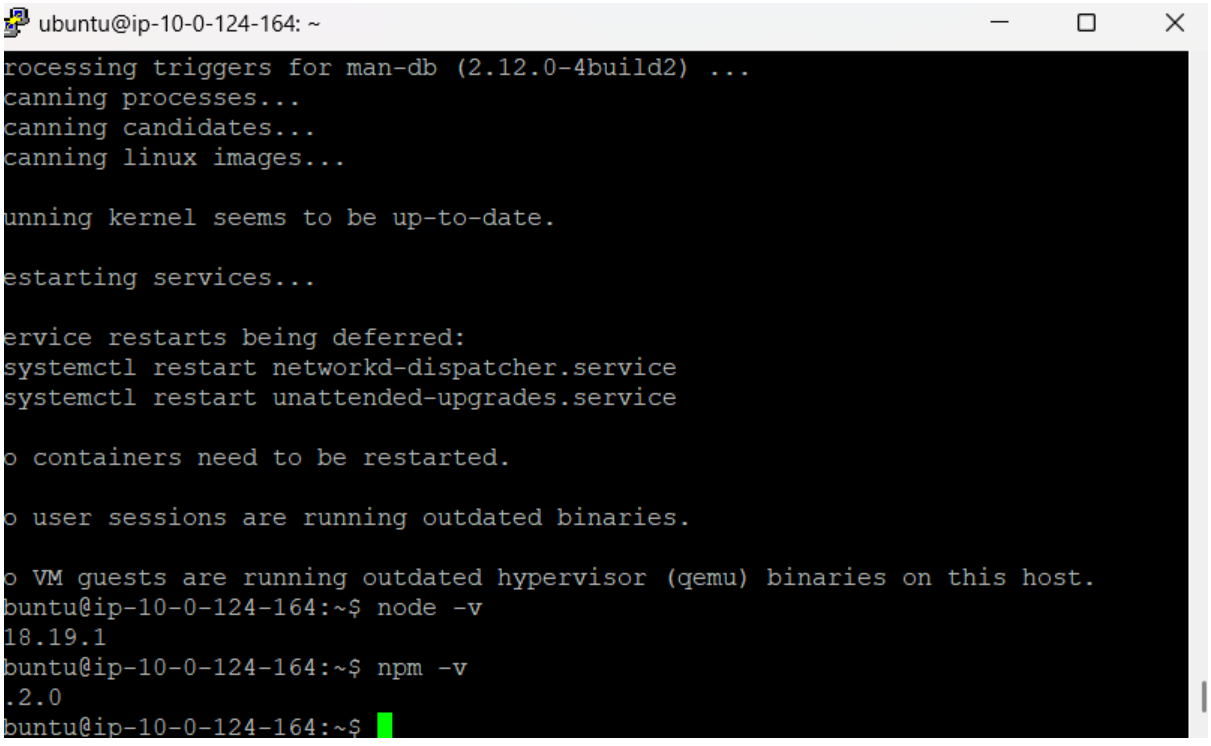
Installs the Flask framework, enabling the creation and execution of the backend web application.

Application Execution

python3 app.py

Runs the Flask application using the Python interpreter. This command starts the application server, allowing it to listen for incoming requests from the web tier and interact with the database as required.





Amazon RDS (MySQL)

What is Amazon RDS?
Amazon Relational Database Service (RDS) is a fully managed database service that simplifies the setup, operation, and scaling of relational databases in the AWS Cloud. AWS handles routine database tasks such as provisioning, patching, backups, monitoring, and recovery, allowing developers to focus on application logic instead of database administration.

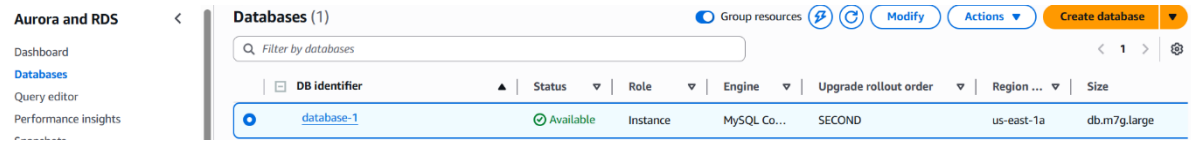
Why Amazon RDS is Used in This Project

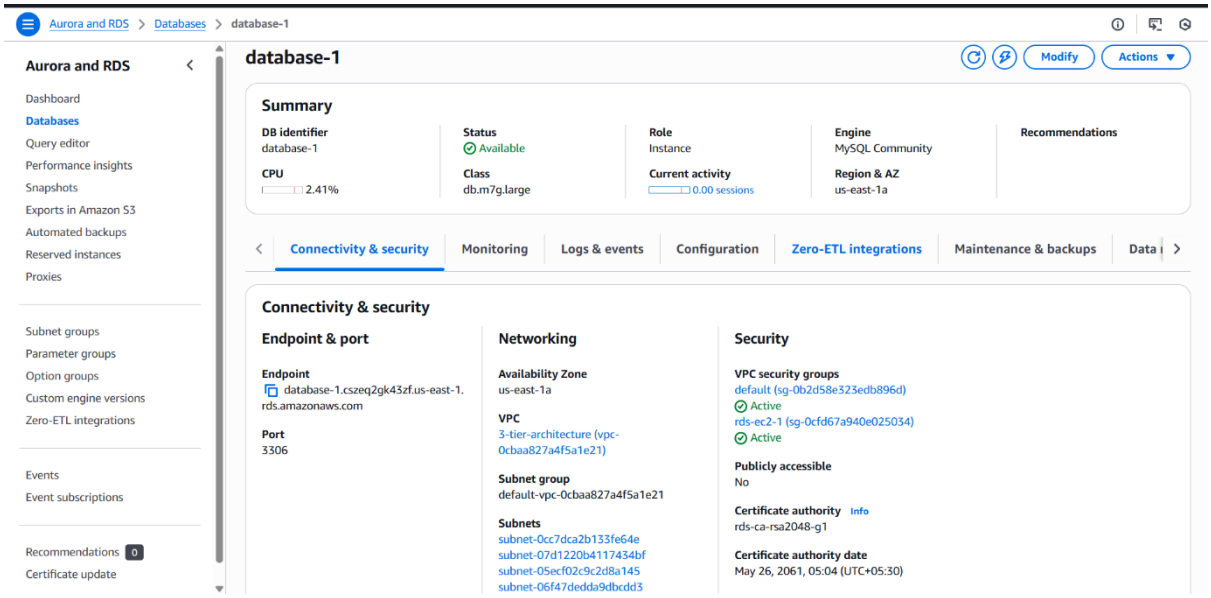
- **Automatic Backups:** RDS automatically performs backups and enables point-in-time recovery, ensuring data protection and easy restoration.
- **High Availability:** Supports Multi-AZ deployments, providing automatic failover in case of infrastructure failure.
- **Enhanced Security:** Integrated with VPC, security groups, IAM, and encryption for data protection.
- **No Server Management:** Eliminates the need to manage operating systems, database patching, or hardware maintenance.

Configuration in This Project

- **Database Engine:** MySQL
- **Network Placement:** Deployed in private subnets within the VPC
- **Public Access:** Disabled to prevent direct internet exposure
- **Access Control:** Only the Application Tier EC2 instances can connect via security group rules

This setup ensures that the database layer remains secure, highly available, and isolated, aligning with best practices for a production-grade three-tier architecture.





MySQL Client Connectivity

The **MySQL Client** is installed on the Application Tier EC2 instances to enable secure communication with the **Amazon RDS (MySQL)** database. Since the database resides in private subnets, direct access from the internet is blocked, and only the application tier is permitted to connect through controlled security group rules.

Why MySQL Client is Required

The MySQL client allows the backend Flask application to:

- Establish a connection with the Amazon RDS MySQL instance.
- Execute SQL queries such as SELECT, INSERT, UPDATE, and DELETE.
- Validate database connectivity during deployment and troubleshooting.

Commands Used

sudo apt install mysql-client -y

Installs the MySQL client package on the Ubuntu EC2 instance, enabling database connectivity tools.

mysql -h <RDS-ENDPOINT> -u admin -p

Connects to the RDS MySQL database using the endpoint provided by AWS, the database username, and a password. Successful login confirms that networking, security groups, and database configuration are correct.

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-10-0-124-164:~$ mysql -h database-1.cszeq2gk43zf.us-east-1.rds.amazona
ws.com -u admin -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 41
Server version: 8.0.43 Source distribution

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
4 rows in set (0.00 sec)

mysql>
```

Validation and Testing

After deploying the three-tier architecture, each layer was tested to ensure proper functionality and connectivity.

Web Tier Test

- Command:

Copy public instance ip and upload to browser

curl http://localhost

- Purpose: Confirms that **Nginx** is running and serving the static website content on the web-tier EC2 instance.

Application Tier Test

- Command:

curl http://localhost:5000

- Purpose: Verifies that the **Flask application** is running on the application-tier EC2 instance and can process dynamic requests.

Database Tier Test

- Command (inside MySQL client):

SHOW DATABASES;

- Purpose: Confirms that the **RDS MySQL database** is accessible from the application tier and that the database connection is correctly established.

End-to-End Request Flow (Three-Tier Architecture)

1. A user sends an HTTP request through a web browser.
2. The request reaches the **Web Tier EC2 instance** located in a public subnet.
3. **Nginx** serves the static frontend content (HTML and CSS).
4. Dynamic requests are forwarded to the **Application Tier**.
5. The **Flask application** processes the request and sends a query to the **MySQL database** hosted on Amazon RDS.
6. Amazon **RDS** processes the query and returns the requested data.
7. The Flask application formats the response.
8. The response is passed back through Nginx to the user's browser.

This flow demonstrates clear separation of concerns, secure communication between tiers, and adherence to AWS best practices for scalability and security.

Project Outcome

- **Successful Deployment:** The secure Three-Tier Architecture was fully deployed across web, application, and database layers.
- **Networking Best Practices:** Implemented VPC with public and private subnets, route tables, Internet Gateway, and NAT Gateway to ensure secure and controlled traffic flow.
- **Scalability, Security, and Isolation:** Auto Scaling Groups provide automatic scaling and fault tolerance, security groups control access between tiers, and database is isolated in private subnets.
- **Full Application Flow Validated:** End-to-end testing confirmed that the web tier serves content, the application tier processes requests, and the database responds correctly, demonstrating a fully functional three-tier system.

Conclusion

This project demonstrates the deployment of a **real-world, production-ready AWS architecture** using core cloud services. It showcases the ability to design and implement a secure, scalable, and highly available three-tier application. Through this project, a strong understanding of **networking (VPC, subnets, route tables, IGW